Northern University of Business and Technology Khulna
Department of Computer Science and Engineering

WEEK 13-14: PRESENTATION & VIVA PREPARATION GUIDE

Project Title: AI-Powered Resume Analyzer and Career Copilot (RESUMIND)

Course Title: Mobile Computing Lab

Course Code: CSE4204

Submitted to:
Md. Riaz Mahmud
Assistant Professor
Department of Computer Science and Engineering
Northern University of Business and Technology, Khulna

Submission Date: 22 October 2026

TEAM INFORMATION SHEET

Field	Details
Official Team Name	CSE4204-8A-T05
Section	8A
Project Title	AI-Powered Resume Analyzer and Career Copilot (RESUMIND)
Team Member List with IDs	1. Mahammad Hasan (ID: 11220320828) - Team Leader / Frontend Developer + UI/UX Designer 2. Shanawaz Sakib (ID: 11220320916) - Core System Integration Engineer 3. Ashikur Rahman Himel (ID: 11220320833) - Database Storage & Quality Controller 4. Rasel Ratul (ID: 11220320827) - Backend Developer
Live Application Link	https://ai-resume-analyzer-psi-lime.vercel.app
GitHub Repository Link	https://github.com/mahamud894/AI_Resume_Analyzer
Figma Link	https://www.figma.com/design/Ls0B3DIjxRKrP2RIzYQxPG/AI-Resume-Analyzer?node-id=2-2&t=lFEVQCoUHt0JSumI-1

1. Presentation Plan

Total time about 12–15 minutes: 7–8 minutes of slides, 5 minutes of live demo, then questions. Every member presents the part they built.

Table 1: Who presents what

Slides	Topic	Presenter	Time
1–3	Title, problem statement, objectives	Mahammad Hasan	2 min
4–5	Features and system architecture	Mahammad Hasan	1.5 min
6	Database design	Ashikur Rahman Himel	1 min
7–8	AI workflow and AI safeguards	Shanawaz Sakib	2 min
9	Security, authentication and API	Rasel Ratul	1 min
10–11	Testing and deployment	Ashikur Rahman Himel / Rasel Ratul	1 min
12	Live demonstration	Mahammad Hasan (driving), Shanawaz Sakib (narrating)	5 min
13–15	Challenges, future work, conclusion	Mahammad Hasan	1.5 min

2. Live Demo Script

Preparation: Before the session: open the /api/health link to wake the backend, log in to the demo account in one tab, keep the resume PDF and the job description on the desktop, and have the slides with backup screenshots open.

Table 2: Demo flow (≈ 5 minutes)

#	Show	Say	Covers
1	Open the live URL on the projector; show it on a phone too	"RESUMIND is live on Vercel and works on any device."	Deployment, responsive
2	Sign up a new account (or Sign in with Google), then log out and log in	"Passwords are bcrypt-hashed; the server returns a JWT valid for 7 days."	Authentication
3	Upload Resume → fill job title and description → choose the PDF → Analyze Resume	"The PDF is checked, stored in GridFS, its text extracted and sent to Gemini in JSON mode."	Main feature, AI
4	Result page: overall gauge, ATS card, open one category	"Every score has good points and improvements with explanations."	AI output

#	Show	Say	Covers
5	Improve My CV (Europass) — open the prepared one if time is short	"The AI never invents facts; missing data becomes a placeholder. Here is the list of every change."	AI feature 2
6	Edit one line in the CV, then Score my improved CV	"Same job, same prompt — the score went from X to Y."	Before/after
7	Generate Cover Letter → Download PDF	"A one-page European business letter using only facts from the CV."	AI feature 3
8	Dashboard history; GitHub Actions green tick	"48 automated tests run on every push before deployment."	Testing, CI

2.1 If something goes wrong

- **Backend sleeping:** talk through the architecture slide for 50 seconds while it wakes.
- **Gemini busy (503):** retry once; otherwise open the prepared analysis in the demo account.
- **No internet:** use the backup screenshots in the slides and the local Docker setup.

3. Expected Viva Questions and Answers

Short, honest answers work best. If you do not know, say how you would find out.

Architecture

Question	Answer
Explain the architecture of your system.	Three tiers. A React single-page app on Vercel talks over HTTPS/JSON with a JWT to an Express 5 REST API on Render. Only the API reaches MongoDB Atlas (data + GridFS files) and Google Gemini, so secrets stay on the server.
Why did you move away from Puter.js?	Puter.js kept everything in the browser. The course needs our own backend, a database that matches the ERD and control over validation and security. Moving AI to the server also hides the API key and lets us validate the model's output.
Why Express and not a framework like NestJS?	Express is small, well-documented and enough for 21 endpoints. The team already knew JavaScript, so frontend and backend share one language.
How is the backend organised?	routes → middleware (auth, validation, upload, rate limit) → controllers → services (pdf, Gemini, Europass, cover letter) → Mongoose models. Errors go to one central error handler.
What happens when the user clicks Analyze Resume?	Upload: Multer checks type, size and %PDF- bytes, files go to GridFS. Analyze: JWT verified, input validated, AI quota checked, pdf-parse extracts text, Gemini returns JSON, we clamp/normalise it, save an Analysis and return 201.

Database design

Question	Answer
Why MongoDB instead of MySQL?	Our main data is nested and variable: AI feedback with tip arrays, a whole Europass CV, a structured letter. Documents store that naturally. We still keep separate collections with references, so it follows the ERD (we also provide schema.sql as the relational equivalent).
Describe your collections and relationships.	users 1–N resumes 1–N analyses 1–0..1 improvedresumes 1–0..1 coverletters. Files are in GridFS (uploads.files / uploads.chunks) and referenced by fileId and imageId.
How do you enforce one improved resume per analysis?	A unique index on improvedresumes.analysisId (and on coverletters.improvedResumeId). Regenerating replaces the document.
What happens to data when a resume is deleted?	Cascade delete in the controller: its analyses, improved resumes, cover letters and both GridFS files are removed. Deleting the account removes everything owned by the user.
Why GridFS instead of saving files on disk?	Render's disk is temporary and is wiped on every deploy. GridFS keeps the files in Atlas next to the data, so they survive redeploys and backups include them.
How do you stop a user reading someone else's data?	Every query filters by userId from the verified token. Another user's id returns 404, so ids cannot even be confirmed to exist.

AI workflow

Question	Answer
Which AI model do you use and how?	Google Gemini (gemini-flash-latest) through the @google/genai SDK in JSON mode: responseMimeType application/json plus a responseJsonSchema, so the reply always has our exact structure.
How do you make the AI output reliable?	Strict schema, then code validation: clamp scores to 0–100, drop malformed tips, normalise dates. Temporary errors (500/503/504) are retried up to 3 times with 2 s and 4 s pauses.
How do you prevent hallucination in the Europass CV?	The prompt forbids inventing facts, and a post-check in code replaces any number not in the original resume with a placeholder, forces MM/YYYY dates, writes qualifications in full and removes personal data such as religion or date of birth.
How is the before/after score calculated?	The improved CV (user's edited version if saved) is converted to text, placeholders are removed, and it is scored with the same prompt and the same job as the original. Stored as afterFeedback.

Question	Answer
How is the cover letter generated?	Gemini writes only opening, two body paragraphs, closing and highlights. The backend adds sender, date, recipient, subject, salutation and sign-off. If the body is not 250–400 words we ask Gemini once more.
What if Gemini is down or the key runs out?	The user gets a clear message: 429 rate limit, 502 model error or 503 busy/missing key. Nothing is saved, and failed requests do not use the user's quota.
How do you control AI cost?	15 AI requests per user per hour, resume text capped at 30 000 characters, and results are saved so viewing them again needs no AI call.

APIs

Question	Answer
How many endpoints do you have?	21: health, 6 auth, 6 resume and 8 AI-related (analyze, improve, get/edit improved, score, generate/get/edit cover letter).
How do you validate input?	express-validator rules for every body and URL parameter (lengths, e-mail, ObjectId). Invalid input returns 400 with a list of fields.
What status codes do you use?	200/201 success, 400 invalid input, 401 not logged in, 404 not found or not yours, 409 duplicate, 413 too large, 422 unreadable PDF, 429 rate limit, 502 AI error, 503 AI unavailable.
How does file upload work?	multipart/form-data with Multer in memory storage, PDF only, 10 MB max, magic-byte check, then streamed into GridFS. The frontend creates the page-1 preview PNG with pdf.js.
How did you test the API?	48 Vitest + Supertest tests with an in-memory MongoDB and a mocked Gemini, run in GitHub Actions on every push.

Authentication and security

Question	Answer
How are passwords stored?	Hashed with bcrypt; never stored or logged in plain text. Minimum 8, maximum 72 characters (bcrypt's limit).
How does JWT authentication work?	After login the server signs a token with JWT_SECRET (7 days). The client sends it as Authorization: Bearer. requireAuth verifies signature and expiry and loads the user.
How do you log out other devices after a password change?	We save passwordChangedAt; the middleware rejects any token issued before that time.

Question	Answer
How does Google Sign-In work?	The Google button gives the browser an ID token. The backend verifies it with google-auth-library against our client id, requires a verified e-mail, then finds or creates the user and returns our own JWT.
How do you protect against brute force?	express-rate-limit: 20 auth requests per 15 minutes per IP; TRUST_PROXY is set on Render so the real client IP is used.
Other security measures?	helmet headers, CORS limited to the frontend URL, 1 MB JSON limit, secrets only in environment variables, .env git-ignored, ownership checks, no stack traces in production errors.

Deployment

Question	Answer
Where and how is it deployed?	Frontend on Vercel (static SPA), backend on Render (render.yaml Blueprint), database on MongoDB Atlas. Every push to main runs CI and redeploys both. Live: https://ai-resume-analyzer-psi-lime.vercel.app
Why is the first request slow?	Render's free plan sleeps after 15 minutes idle; waking takes about 50 seconds. A paid plan or a keep-alive ping would remove that.
Where are secrets kept?	Locally in .env (git-ignored); in production in the Render and Vercel dashboards. JWT_SECRET is generated by Render.

4. Individual Talking Points

Each member must be able to explain their own work in about one minute and answer follow-up questions on it.

Mahammad Hasan (11220320828) — Team Leader / Frontend Developer + UI/UX Designer

- Team leader: planning, weekly tasks, GitHub workflow.
- React Router 8 structure, protected routes and the Zustand auth store.
- lib/api.ts: one fetch wrapper that adds the JWT and handles 401.
- All pages and the RESUMIND design (Tailwind CSS 4, mobile-first).
- Europass CV editor (inline editing, reset), cover letter page, report PDF, score chart.

Shanawaz Sakib (11220320916) — Core System Integration Engineer

- Gemini service: JSON mode, responseJsonSchema, retries.

- Prompts for analysis, Europass rewrite and cover letter.
- End-to-end flow PDF → text → Gemini → saved feedback → UI.
- Post-check rules that stop invented numbers and dates.
- AI rate limit (15/hour/user) and Docker Compose setup.

Ashikur Rahman Himel (11220320833) — Database Storage & Quality Controller

- ERD and the five Mongoose models; indexes (unique, sparse).
- GridFS storage of PDFs and preview images.
- Seed data and the schema documents (mongodb-schema.md, schema.sql).
- 48 automated tests with Vitest, Supertest and mongodb-memory-server.
- CI workflow in GitHub Actions; dependency security audit.

Rasel Ratul (11220320827) — Backend Developer

- Express 5 app, routes, controllers and central error handler.
- Register/login with bcrypt and JWT; Google Sign-In verification.
- Multer upload with size, type and %PDF- checks.
- helmet, CORS, rate limiting, ownership checks.
- Change password (logs out other devices), delete account; Render deployment.

5. Challenges We Faced

Challenge	How we solved it
Gemini sometimes returned malformed or inconsistent JSON	JSON mode with a strict schema plus validation/clamping in code
AI invented percentages and dates in the CV	Deterministic post-check that turns new numbers into placeholders
Gemini "high demand" 503 errors	Automatic retries with back-off and clear user messages
Uploaded files lost after a Render redeploy	Moved file storage to MongoDB GridFS
Same rate limit for everyone behind Render's proxy	TRUST_PROXY=1 so each visitor's real IP is counted
Out-of-sync package-lock broke npm ci	Regenerated the lock file and added CI so it is caught before deploy
Making a document-heavy app usable on phones	Mobile-first layout, stacked result page, A4 sheets that scale

6. Limitations to Admit Honestly

- Scanned (image-only) PDFs cannot be read yet — OCR is future work.
- AI scores may vary by a few points between runs.
- Free hosting: cold start of about 50 seconds and limited Gemini quota.
- English only.

7. Academic Integrity Statement

The first prototype followed a public YouTube tutorial (JavaScript Mastery, "AI Resume Analyzer") that used Puter.js. The team acknowledges this source. Everything beyond it was designed and built for this course: the Express backend, the MongoDB design, authentication, the Europass rewrite with post-checks, before/after scoring, the cover letter generator, tests, CI and deployment. AI coding assistants were used as a tool during development; every member reviewed, understood and can explain the code in their area.